



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего
образования
«Московский государственный технический университет имени
Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»

КАФЕДРА «СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ»

Лабораторная работа №2

«Объектно-ориентированные возможности языка Python»

ДИСЦИПЛИНА: «Парадигмы и конструкции языков программирования»

Выполнил: студент гр. ИУ5-34Б

(Подпись) (Лимаренко Д. А.)
(Ф.И. О.)

2026 г.

Задание:

1. Необходимо создать виртуальное окружение и установить в него хотя бы один внешний пакет с использованием `pip`.
2. Необходимо разработать программу, реализующую работу с классами. Программа должна быть разработана в виде консольного приложения на языке Python 3.
3. Все файлы проекта (кроме основного файла `main.py`) должны располагаться в пакете `lab_python_oop`.
4. Каждый из нижеперечисленных классов должен располагаться в отдельном файле пакета `lab_python_oop`.
5. Абстрактный класс «Геометрическая фигура» содержит абстрактный метод для вычисления площади фигуры.
6. Класс «Цвет фигуры» содержит свойство для описания цвета геометрической фигуры.
7. Класс «Прямоугольник» наследуется от класса «Геометрическая фигура». Класс должен содержать конструктор по параметрам «ширина», «высота» и «цвет». В конструкторе создается объект класса «Цвет фигуры» для хранения цвета. Класс должен переопределять метод, вычисляющий площадь фигуры.
8. Класс «Круг» создается аналогично классу «Прямоугольник», задается параметр «радиус». Для вычисления площади используется константа `math.pi` из модуля [math](https://docs.python.org/3/library/math.html).
9. Класс «Квадрат» наследуется от класса «Прямоугольник». Класс должен содержать конструктор по длине стороны. Для классов «Прямоугольник», «Квадрат», «Круг»:
 - Определите метод "getr", который возвращает в виде строки основные параметры фигуры, ее цвет и площадь. Используйте метод `format` - <https://pyformat.info/>
 - Название фигуры («Прямоугольник», «Квадрат», «Круг») должно задаваться в виде поля данных класса и возвращаться методом класса.
10. В корневом каталоге проекта создайте файл `main.py` для тестирования Ваших классов (используйте следующую конструкцию

- https://docs.python.org/3/library/_main_.html). Создайте следующие объекты и выведите о них информацию в консоль (N - номер Вашего варианта по списку группы):

- Прямоугольник синего цвета шириной N и высотой N.
- Круг зеленого цвета радиусом N.
- Квадрат красного цвета со стороной N.
- Также вызовите один из методов внешнего пакета, установленного с использованием pip.

11. Дополнительное задание. Протестируйте корректность работы Вашей программы с помощью модульного теста.

N=15

Выполнение работы

1. Абстрактный класс геометрической фигуры

Файл lab_python_oop/shape.py

```
from abc import ABC, abstractmethod

class Shape(ABC):
    """Абстрактный класс геометрической фигуры."""

    @abstractmethod
    def area(self):
        """Вычислить площадь фигуры."""
        pass
```

2. Класс цвета фигуры

Файл lab_python_oop/color.py

```
class Color:
    """Цвет геометрической фигуры."""

    def __init__(self, color):
        self.color = color

    @property
    def color(self):
        """Получить цвет."""
        return self._color

    @color.setter
    def color(self, value):
        """Изменить цвет."""
        self._color = value
```

3. Класс прямоугольника

Файл lab_python_oop/rectangle.py

```
from lab_python_oop.shape import Shape
from lab_python_oop.color import Color

class Rectangle(Shape):
    """Прямоугольник."""

    name = "Прямоугольник"

    def __init__(self, width, height, color):
        self.width = width
        self.height = height
        self.color = Color(color)

    def area(self):
        return self.width * self.height

    @classmethod
    def get_name(cls):
        return cls.name

    def __repr__(self):
        return (
            "{}: ширина = {}, высота = {}, "
            "цвет = {}, площадь = {:.2f}"
        ).format(
            self.get_name(),
            self.width,
            self.height,
            self.color.color,
            self.area()
        )
```

4. Класс круга

Файл lab_python_oop/circle.py

```
import math

from lab_python_oop.shape import Shape
from lab_python_oop.color import Color

class Circle(Shape):
    """Круг."""

    name = "Круг"

    def __init__(self, radius, color):
        self.radius = radius
        self.color = Color(color)

    def area(self):
        return math.pi * self.radius ** 2

    @classmethod
    def get_name(cls):
        return cls.name

    def __repr__(self):
        return (
            "{}: радиус = {}, цвет = {}, площадь = {:.2f}"
            .format(
                self.get_name(),
                self.radius,
                self.color.color,
                self.area()
            )
        )
```

5. Класс квадрата

Файл `lab_python_oop/square.py`

```
from lab_python_oop.rectangle import Rectangle

class Square(Rectangle):
    """Квадрат."""

    name = "Квадрат"

    def __init__(self, side, color):
        super().__init__(side, side, color)

    def __repr__(self):
        return (
            "{}: сторона = {}, цвет = {}, площадь = {:.2f}"
            .format(
                self.get_name(),
                self.width,
                self.color.color,
                self.area()
            )
        )
```

Для организации пакета создан пустой файл `lab_python_oop/__init__.py`.

6. Основная программа

Файл main.py

```
import colorama
from colorama import Fore, Style

from lab_python_oop.rectangle import Rectangle
from lab_python_oop.circle import Circle
from lab_python_oop.square import Square

def main():
    colorama.init(autoreset=True)

    N = 15

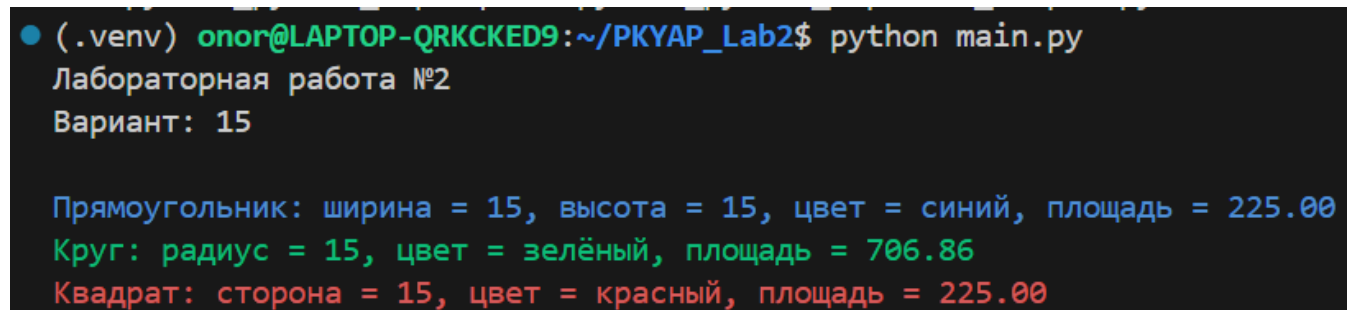
    rectangle = Rectangle(N, N, "синий")
    circle = Circle(N, "зелёный")
    square = Square(N, "красный")

    print("Лабораторная работа №2")
    print("Вариант: {}".format(N))
    print()

    print(Fore.BLUE + repr(rectangle) + Style.RESET_ALL)
    print(Fore.GREEN + repr(circle) + Style.RESET_ALL)
    print(Fore.RED + repr(square) + Style.RESET_ALL)

if __name__ == "__main__":
    main()
```

Экранные формы с примерами выполнения программы



```
● (.venv) onor@LAPTOP-QRKCKED9:~/PKYAP_Lab2$ python main.py
Лабораторная работа №2
Вариант: 15

Прямоугольник: ширина = 15, высота = 15, цвет = синий, площадь = 225.00
Круг: радиус = 15, цвет = зелёный, площадь = 706.86
Квадрат: сторона = 15, цвет = красный, площадь = 225.00
```

Рисунок 1 – Результат выполнения программы

7. Модульные тесты

Файл lab_python_oop/test_shapes.py

```
import math
import unittest

from lab_python_oop.shape import Shape
from lab_python_oop.color import Color
from lab_python_oop.rectangle import Rectangle
from lab_python_oop.circle import Circle
from lab_python_oop.square import Square

class TestShapes(unittest.TestCase):
    """Проверка классов геометрических фигур."""

    def test_shape_is_abstract(self):
        # Объект абстрактного класса создавать нельзя.
        with self.assertRaises(TypeError):
            Shape()

    def test_color_property(self):
        # Проверяем чтение и изменение свойства цвета.
        color = Color("синий")
        self.assertEqual(color.color, "синий")

        color.color = "красный"
        self.assertEqual(color.color, "красный")

    def test_rectangle(self):
        # Разные стороны помогают проверить формулу площади.
        rectangle = Rectangle(4, 6, "синий")

        self.assertIsInstance(rectangle, Shape)
        self.assertIsInstance(rectangle.color, Color)
        self.assertEqual(rectangle.width, 4)
        self.assertEqual(rectangle.height, 6)
        self.assertEqual(rectangle.color.color, "синий")
        self.assertEqual(rectangle.area(), 24)

    def test_circle(self):
        circle = Circle(3, "зелёный")

        self.assertIsInstance(circle, Shape)
        self.assertIsInstance(circle.color, Color)
        self.assertEqual(circle.radius, 3)
        self.assertEqual(circle.color.color, "зелёный")
        self.assertAlmostEqual(circle.area(), 9 * math.pi)

    def test_square(self):
        square = Square(5, "красный")

        self.assertIsInstance(square, Rectangle)
        self.assertIsInstance(square.color, Color)
        self.assertEqual(square.width, 5)
        self.assertEqual(square.height, 5)
        self.assertEqual(square.color.color, "красный")
        self.assertEqual(square.area(), 25)

    def test_class_names(self):
        self.assertEqual(Rectangle.get_name(), "Прямоугольник")
        self.assertEqual(Circle.get_name(), "Круг")
        self.assertEqual(Square.get_name(), "Квадрат")
```

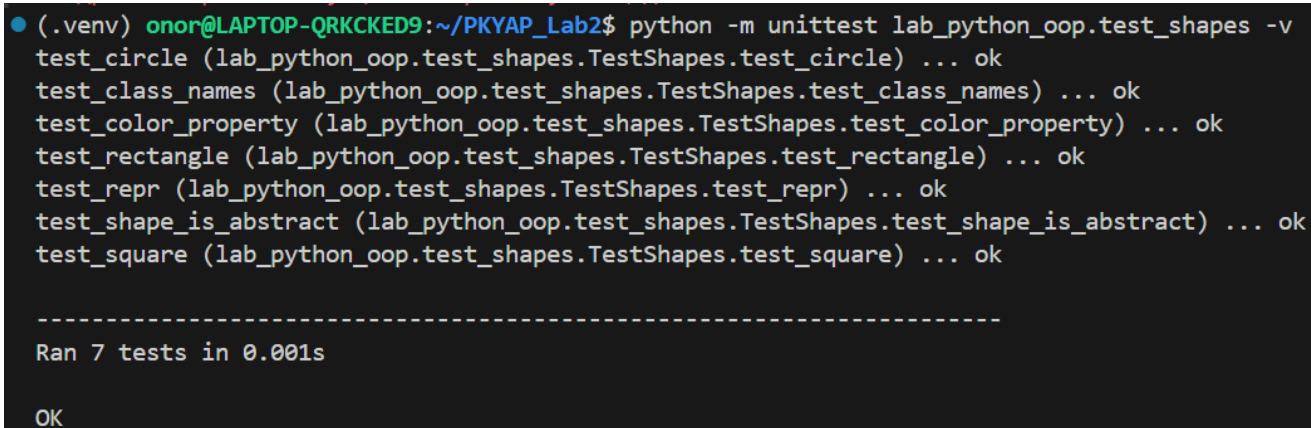
```

def test_repr(self):
    self.assertEqual(
        repr(Rectangle(4, 6, "синий")),
        "Прямоугольник: ширина = 4, высота = 6, "
        "цвет = синий, площадь = 24.00"
    )
    self.assertEqual(
        repr(Circle(3, "зелёный")),
        "Круг: радиус = 3, цвет = зелёный, площадь = 28.27"
    )
    self.assertEqual(
        repr(Square(5, "красный")),
        "Квадрат: сторона = 5, цвет = красный, площадь = 25.00"
    )

if __name__ == "__main__":
    unittest.main()

```

Экранные формы с примерами выполнения программы



```

● (.venv) onor@LAPTOP-QRKCKED9:~/PKYAP_Lab2$ python -m unittest lab_python_oop.test_shapes -v
test_circle (lab_python_oop.test_shapes.TestShapes.test_circle) ... ok
test_class_names (lab_python_oop.test_shapes.TestShapes.test_class_names) ... ok
test_color_property (lab_python_oop.test_shapes.TestShapes.test_color_property) ... ok
test_rectangle (lab_python_oop.test_shapes.TestShapes.test_rectangle) ... ok
test_repr (lab_python_oop.test_shapes.TestShapes.test_repr) ... ok
test_shape_is_abstract (lab_python_oop.test_shapes.TestShapes.test_shape_is_abstract) ... ok
test_square (lab_python_oop.test_shapes.TestShapes.test_square) ... ok

-----
Ran 7 tests in 0.001s

OK

```

Рисунок 2 – Результат выполнения модульных тестов